Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

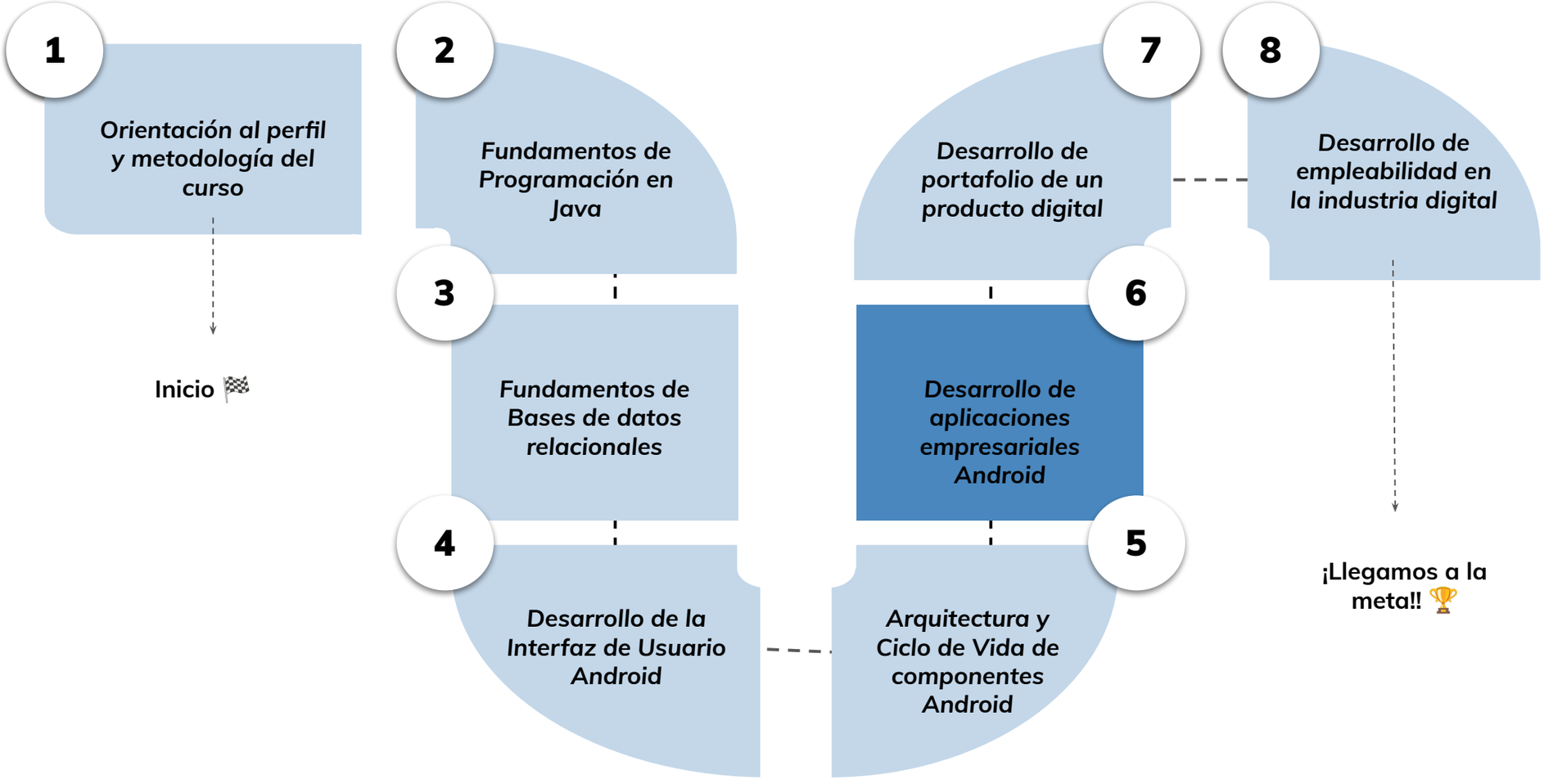
Testing y android -Parte I

AE: Implementar tests unitarios y de instrumentación para la verificación del buen funcionamiento de los componentes de un proyecto android.

alkemy

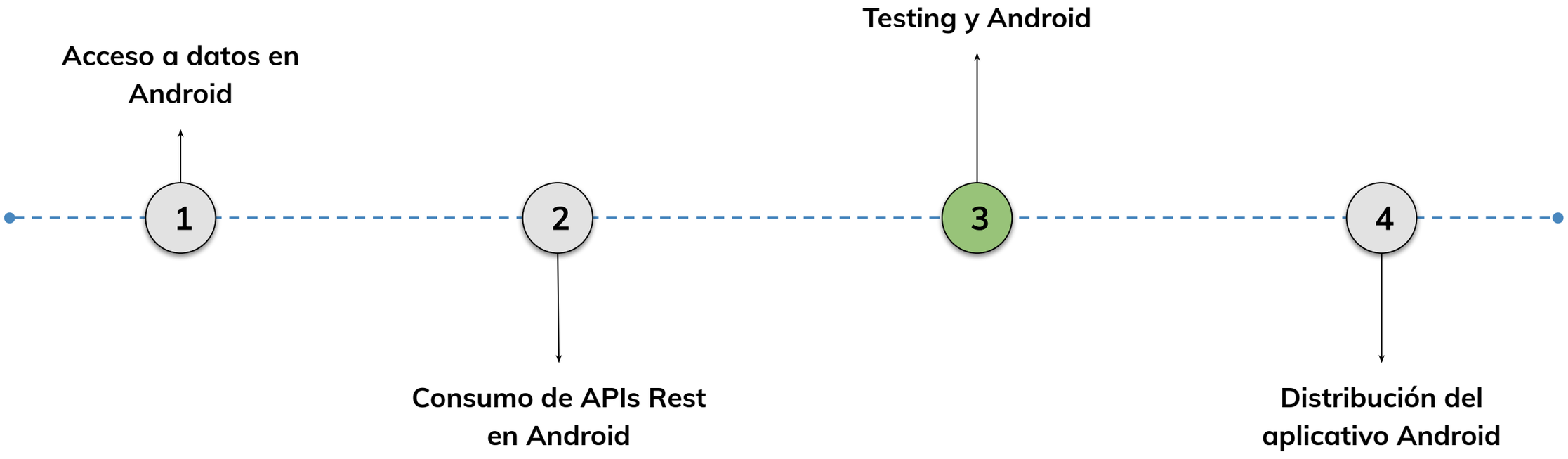
Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



Learning Path

¿Cuáles temas trabajaremos hoy?

6. Start! 🏁

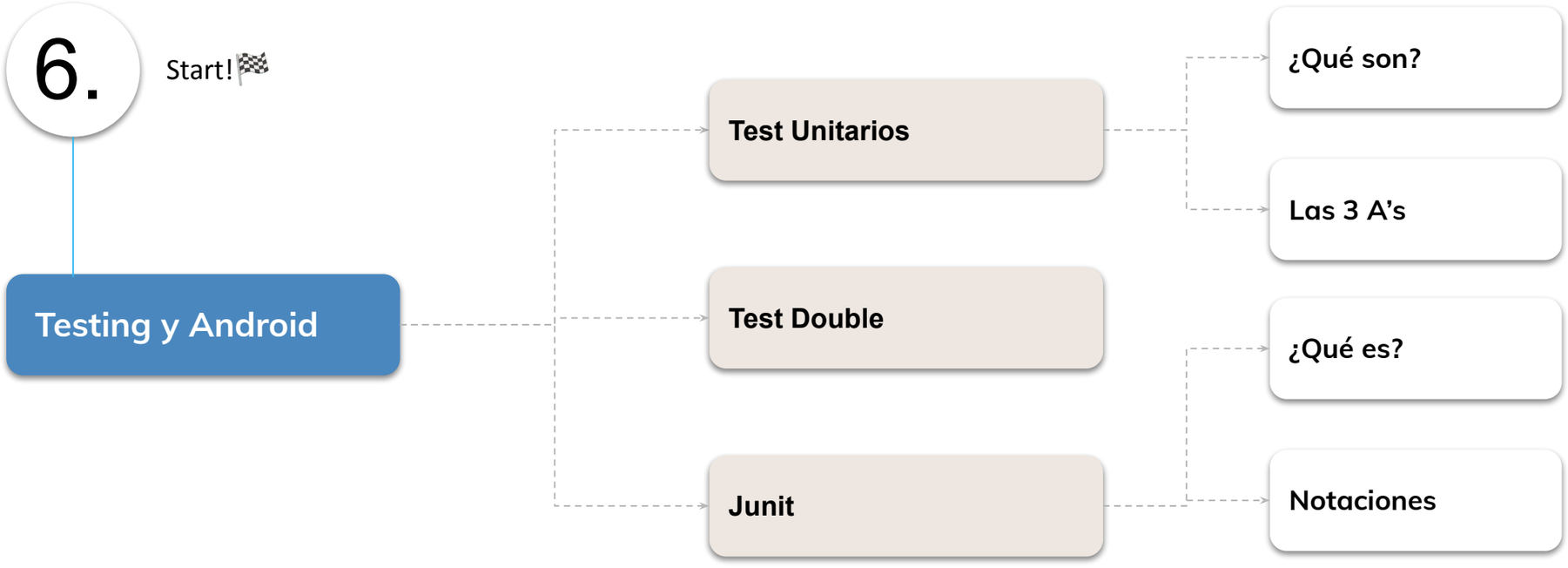
Testing y Android

- Intro a Testing
  - ¿Qué es?
  - ¿Cómo se aplica?
- Tipos de Testing
  - ¿Por qué?
  - Tipos
- Configuración ambiente

alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?



alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- *Entender qué es el testing y el control de calidad*
- *Comprender cuáles son los tipos de pruebas*
- *Ver cómo se configuran los ambientes de pruebas*
- *Comprender qué son los Test Unitarios*
- *Conocer qué es la librería Junit*
- *Ver algunos de los usos de Junit*

Rompehielo🥶

Contexto: 🙌

Cuando hablamos de testeo de calidad de código podemos referirnos a muchas cosas dependiendo del contexto y del ambiente en que estemos trabajando

Consigna: ✍️

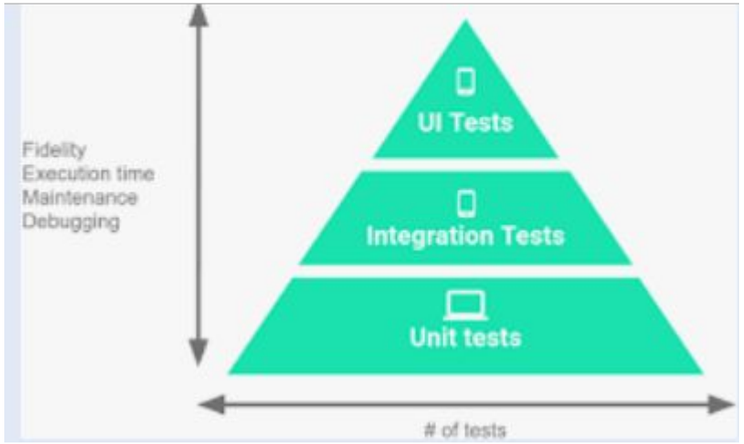
¿Qué cosas creen que tenemos que tener en cuenta en un ambiente donde trabajamos en equipo y tenemos que mantener un grado de calidad general?

› Intro a Testing

Testing

¿Qué es?:

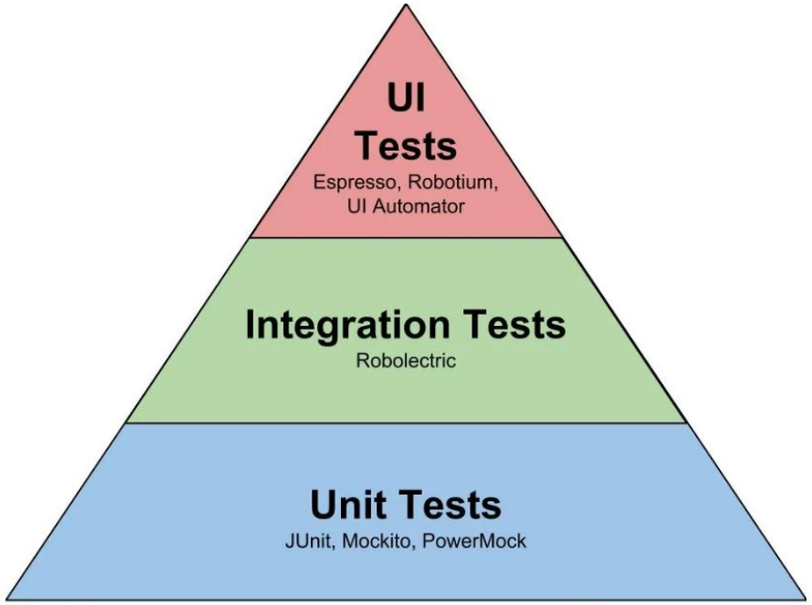
El testing, también conocido como pruebas o pruebas de software, es una práctica esencial en el desarrollo de software que tiene como objetivo verificar y validar que un programa o aplicación funcione correctamente y cumpla con los requisitos establecidos. Consiste en diseñar y ejecutar diferentes escenarios para evaluar el comportamiento del software y detectar posibles errores, defectos o problemas de rendimiento.



Testing y Android

¿Cómo se aplica?:

En el contexto de Android, el testing se relaciona directamente con la creación y ejecución de pruebas para aplicaciones móviles. Las pruebas en Android son fundamentales para garantizar la calidad y el rendimiento de las aplicaciones antes de su lanzamiento al mercado o distribución a los usuarios.



Testing y Android

¿Cómo se aplica?:

Cuando trabajamos en Android tenemos que tener en cuenta cosas diferentes a otras plataformas ya que hay un mayor énfasis en rendimiento y consumo de recursos. También tenemos que tener en cuenta que como nuestro código tiene que correr en múltiples modelos y tipos de dispositivos necesitamos hacer pruebas sobre las vistas y compatibilidades.

› Tipos de Testing

Tipos de Testing

¿Por qué?:

En cualquier ambiente de desarrollo hay muchas cosas para validar: Funcionalidades particulares, Integracion de modulos, vistas y rendimiento general. Por lo que optamos por dividir estas pruebas en diferentes etapas y enfoques para mantener un orden y asignación de responsabilidades claros.

Tipos de Testing

Tests Unitarios:

Son pruebas enfocadas en evaluar unidades individuales del código, como métodos y clases, de manera aislada del resto de la aplicación. Utiliza frameworks de prueba como JUnit y Mockito para escribir pruebas unitarias. Las pruebas unitarias se centran en probar unidades individuales de código, como funciones o métodos.
Asegúrate de que tus funciones y métodos sean lo más independientes posible y puedan ser probados de forma aislada.

Tipos de Testing

Tests de Integración:

Se enfocan en evaluar la interacción y el funcionamiento correcto de diferentes componentes y módulos de la aplicación al trabajar en conjunto.
Estas pruebas permiten detectar problemas de compatibilidad y comunicación entre distintas partes de la aplicación.
Las pruebas de integración evalúan cómo las diferentes partes de tu aplicación interactúan entre sí.
Utiliza frameworks como Espresso para escribir pruebas de interfaz de usuario y pruebas de integración.

Tipos de Testing

Tests de Aceptación:

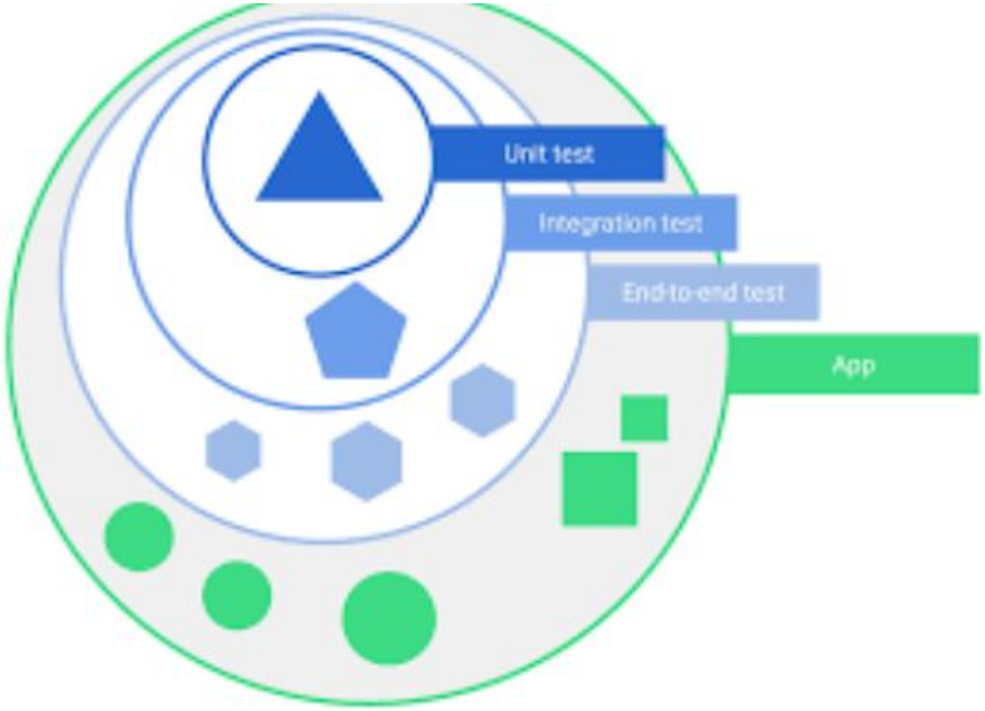
Son pruebas que verifican que la aplicación cumpla con los requisitos y especificaciones establecidas por el cliente o el equipo de desarrollo. Evalúan el comportamiento global de la aplicación para asegurar que todas las funcionalidades principales estén operativas.
Utiliza frameworks como Cucumber o Appium para escribir pruebas de aceptación.

Designed by Freepik

Tipos de Testing

Tests de Interfaz de Usuario (UI):

Se centran en evaluar la experiencia del usuario al interactuar con la interfaz gráfica de la aplicación. Estas pruebas comprueban que los elementos de la interfaz respondan adecuadamente a las acciones del usuario.



Tipos de Testing

Tests de Rendimiento:

Evalúan el rendimiento de la aplicación bajo diferentes condiciones, como carga de datos, respuesta a múltiples solicitudes y consumo de recursos del dispositivo. Estas pruebas buscan identificar posibles cuellos de botella y optimizar el rendimiento de la aplicación.

› Configuración de Testing

Configuración de Testing

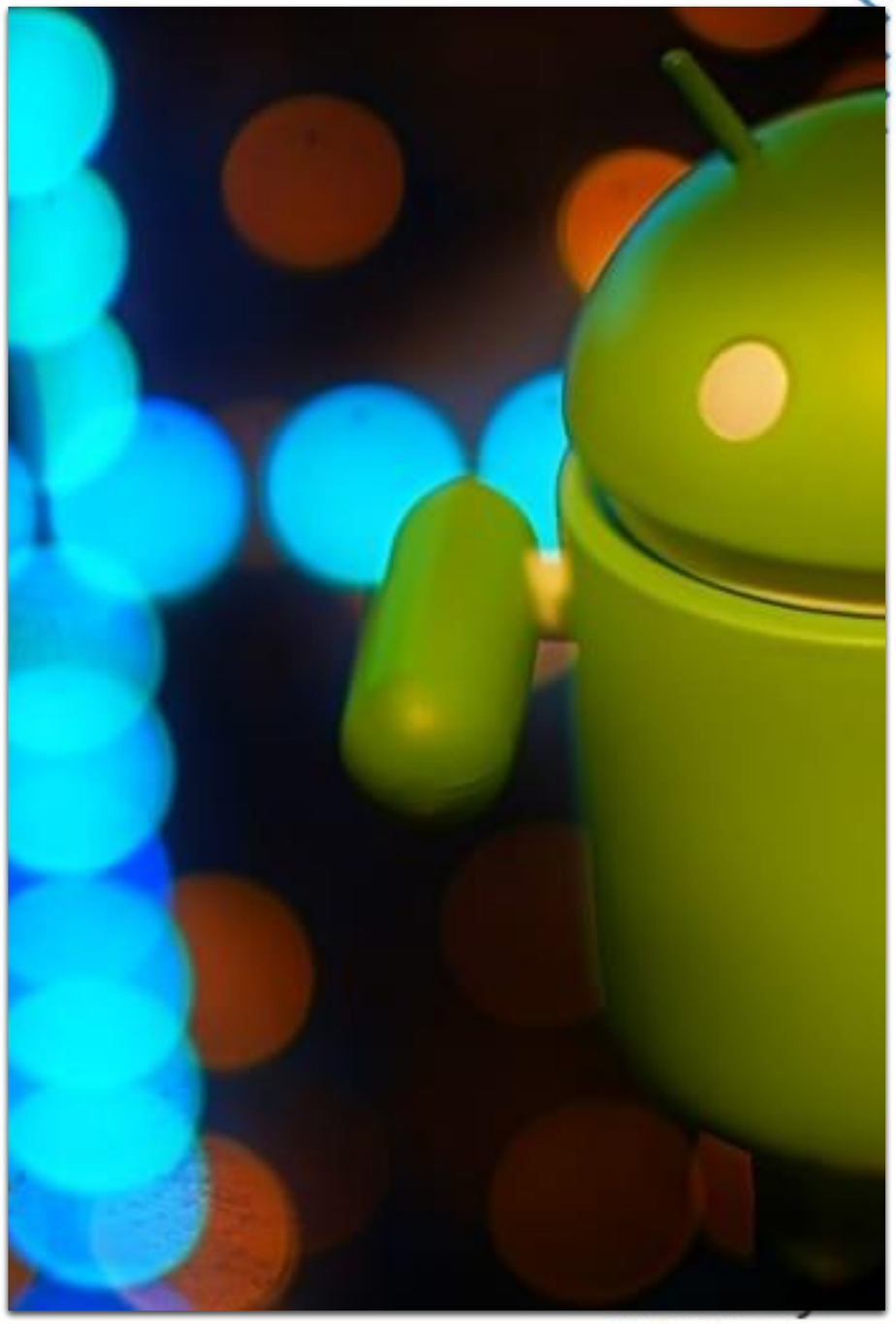
¿Por qué?:

Configurar un ambiente de pruebas en Android implica preparar el entorno de desarrollo y ejecución de pruebas para asegurarte de que tu aplicación funcione de manera correcta y cumpla con los requisitos antes de lanzarla al público.

Tipos de Testing

Configurar Dependencias:

En tu archivo de construcción de Gradle (build.gradle), asegúrate de incluir las dependencias necesarias para pruebas. Las dependencias típicas incluyen JUnit para pruebas unitarias y Espresso para pruebas de interfaz de usuario.



Tipos de Testing

Escribir Pruebas:

En el módulo de pruebas, puedes comenzar a escribir tus pruebas. Utiliza JUnit para pruebas unitarias y Espresso para pruebas de interfaz de usuario. Asegúrate de cubrir diferentes aspectos de tu aplicación, como funciones críticas, casos límite y escenarios de error.

Tipos de Testing

Ejecutar Pruebas:

Puedes ejecutar tus pruebas directamente desde Android Studio. Haz clic con el botón derecho en tu clase de prueba y selecciona "Run". También puedes ejecutar todas las pruebas de tu proyecto de un solo golpe.

¿Alguna consulta?

alkemy

Momento:

Time-out!

⏰5 -10 min.

alkemy

› Test unitarios

Test unitarios

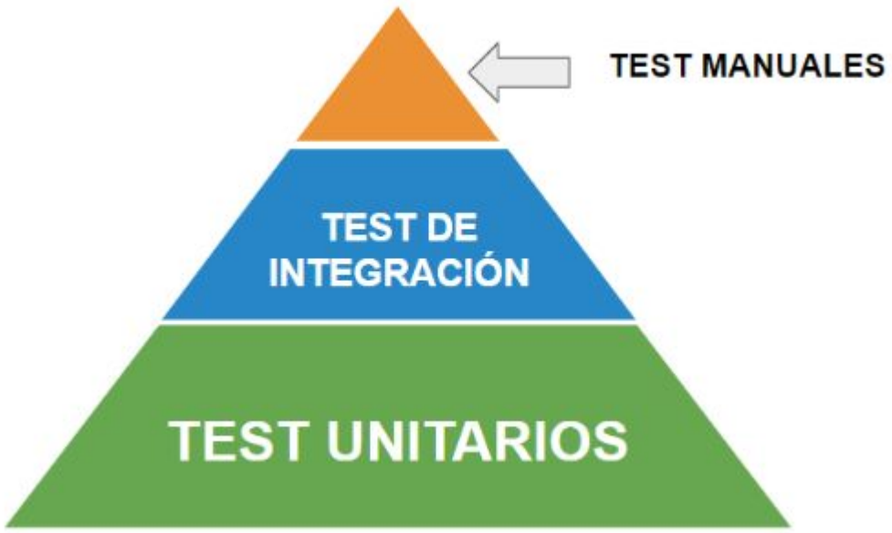
¿Qué son?:

El concepto de pruebas unitarias se basa en la idea de que el código de un programa puede ser descompuesto en unidades más pequeñas, como funciones, métodos o clases. Las pruebas unitarias consisten en aislar una de estas unidades (por ejemplo, una función o un método) y comprobar que su funcionamiento es correcto y que produce los resultados esperados.
Cuando realizamos pruebas unitarias, nos enfocamos en una única "unidad" de código a la vez, separándola del resto del programa y probando su comportamiento de manera individual

Test unitarios

Interactuar:

Se logra hacer test que incluyan otras secciones de código mediante el uso de técnicas como stubs o mocks, que nos permiten simular el comportamiento de otras partes del programa que interactúan con la unidad que estamos probando. De esta manera, garantizamos que las pruebas sean independientes y no estén afectadas por el funcionamiento de otros componentes.



Test unitarios

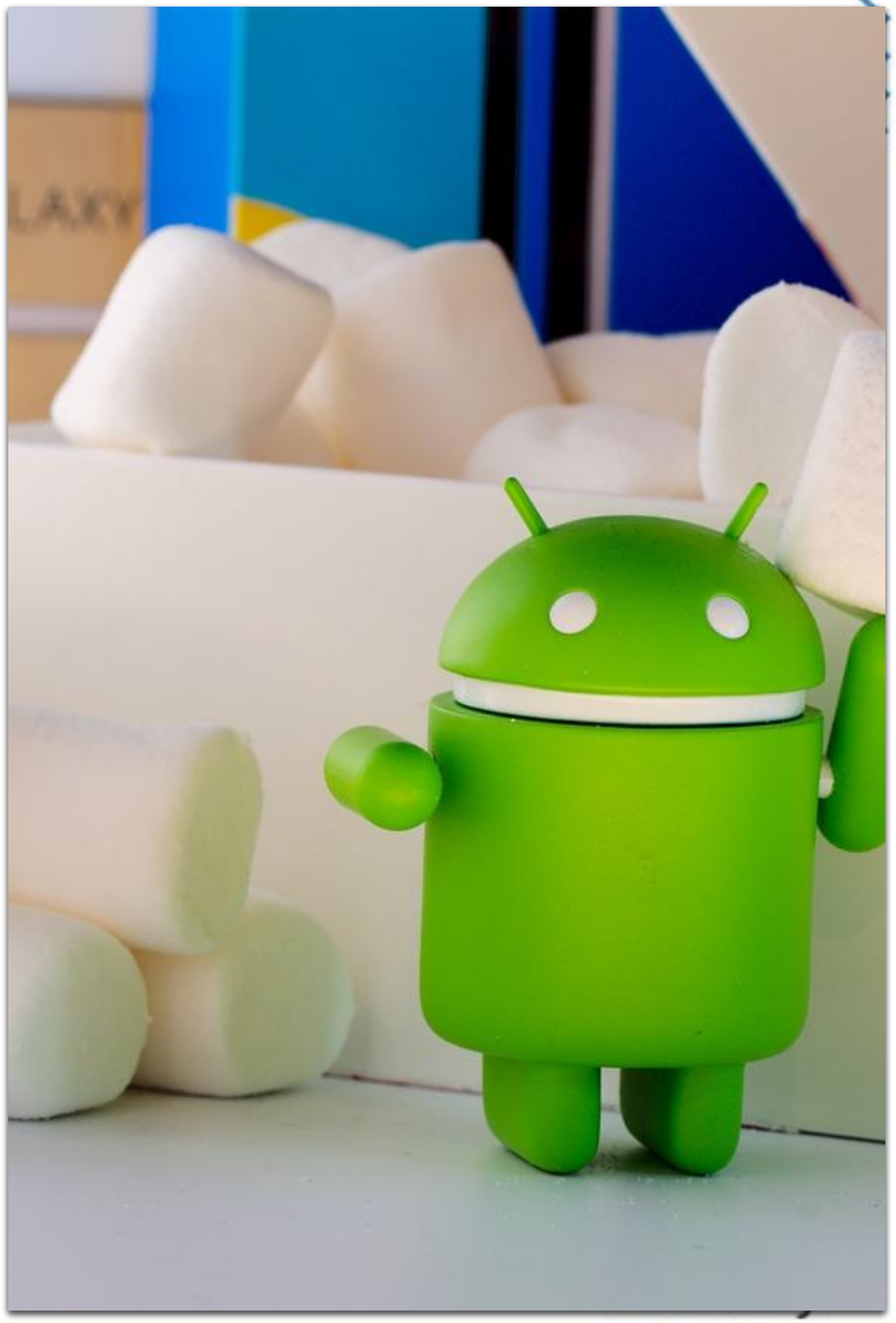
Las 3 A's:

son un concepto que se utiliza para guiar la escritura de pruebas unitarias efectivas y significativas. Estas 3 A's representan los principios clave que deben cumplir las pruebas unitarias para ser consideradas de alta calidad:

Test unitarios

Arrange (Preparar):

En esta etapa, se prepara el escenario para la prueba unitaria. Esto implica configurar el entorno necesario para que la unidad de código que se va a probar se ejecute correctamente. Aquí se definen las entradas y condiciones iniciales que la unidad debe recibir o enfrentar antes de ser probada.



Test unitarios

Act (Actuar)

En esta etapa, se ejecuta la unidad de código que se está probando, utilizando las entradas y condiciones establecidas en la etapa de preparación. Aquí se activa la funcionalidad que queremos evaluar para que realice su trabajo.

Test unitarios

Assert (Afirmar):

En esta etapa, se verifica el resultado o el comportamiento esperado de la unidad de código que fue activada en la etapa anterior. Se comparan los resultados obtenidos con los resultados esperados para determinar si la prueba fue exitosa o no. Si los resultados coinciden con las expectativas, la prueba se considera satisfactoria; de lo contrario, se considera que ha fallado.

› Test Double

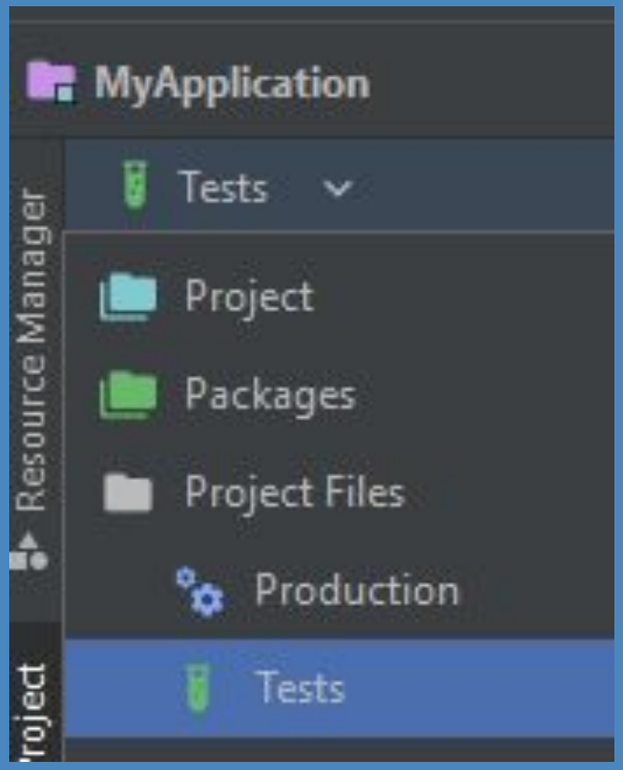
Test Double

¿Qué son?:

Un test double (doble de prueba en español) es un término general que se refiere a cualquier objeto simulado que se utiliza para reemplazar un componente real o dependencia externa durante la ejecución de una prueba unitaria. Los mocks se utilizan para aislar la unidad de código que se está probando y garantizar que la prueba sea independiente y no dependa de otras partes del sistema.

Ubicación

Cuando creamos Test los agregamos a su propia sección en Android Studio donde los almacenamos. Esta es una forma de separarlos del proyecto y poder hacer un mapeo de todas las funcionalidades que queremos mapear

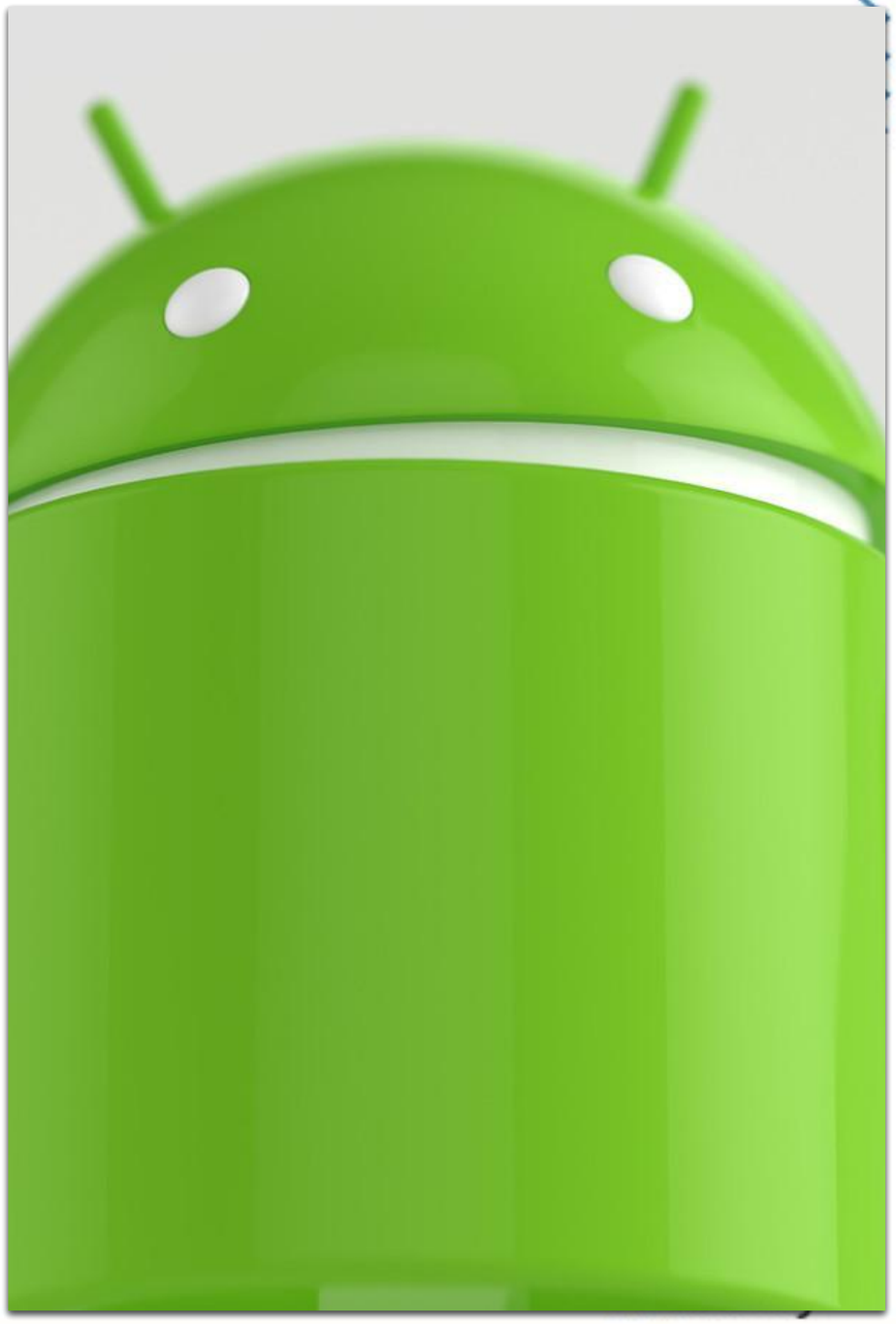


alkemy

Tipos de Testing Double

Dummy:

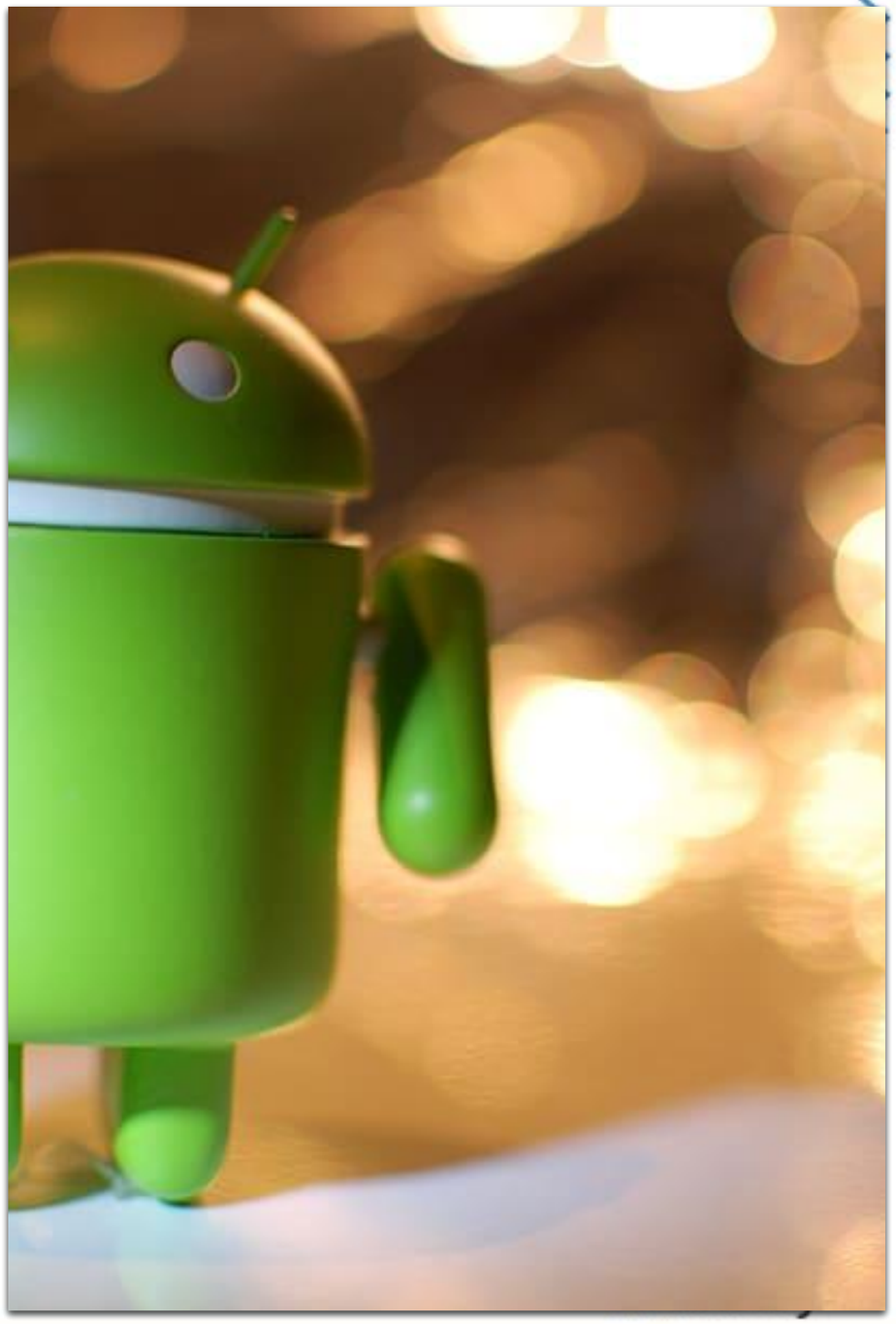
Es un objeto que se pasa como argumento a una función o método durante una prueba, pero su implementación no es relevante para la prueba en sí. En general, los dummies son objetos que no realizan ninguna acción y solo existen para satisfacer la firma de una función o método.



Tipos de Testing Double

Fake:

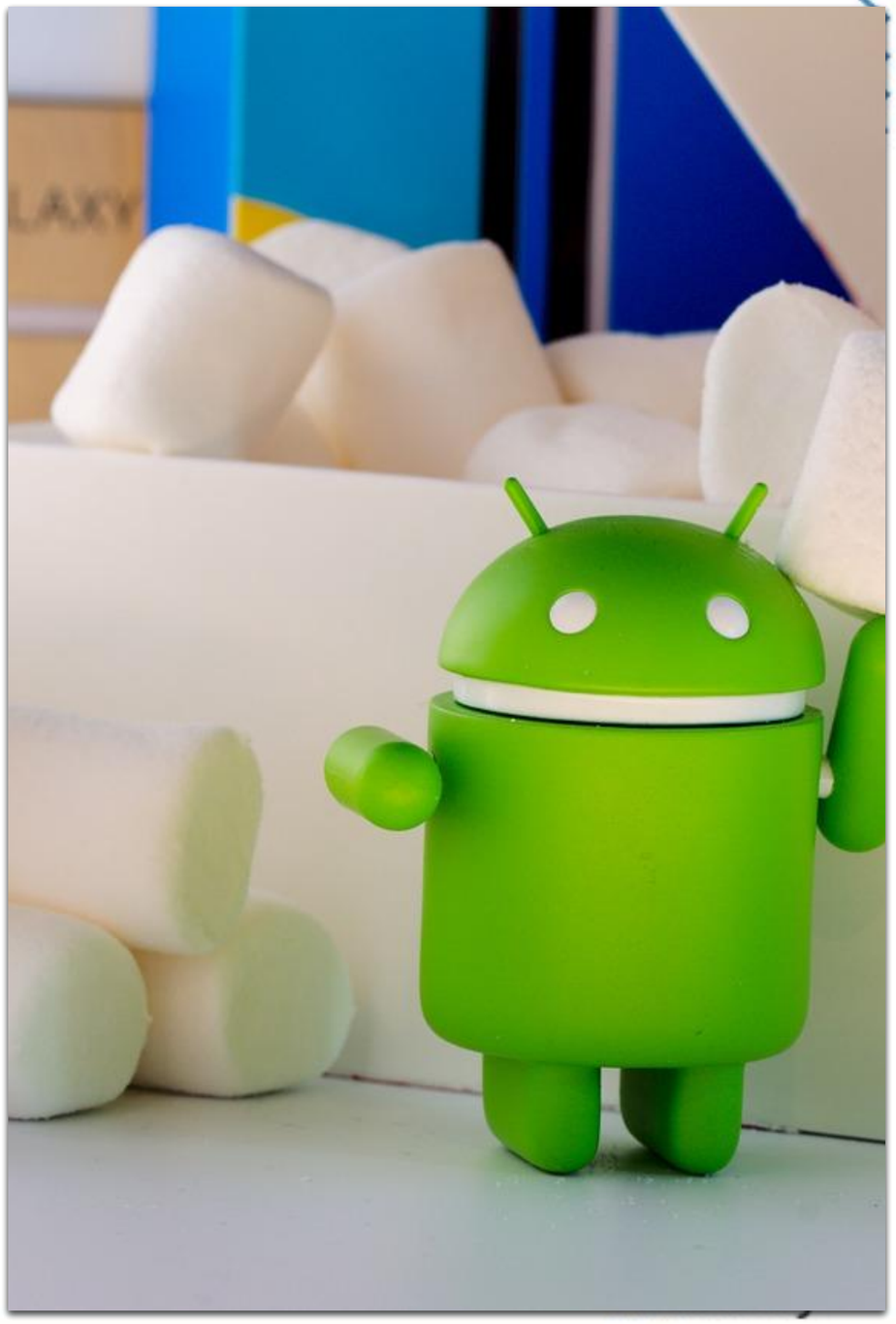
Es un objeto que tiene una implementación simplificada de una dependencia real, pero no es adecuado para producción. Los fakes proporcionan una implementación "bastante real" para las pruebas, pero pueden ser menos eficientes o tener limitaciones en comparación con la implementación real.



Tipos de Testing Double

Stub:

Es un objeto que proporciona respuestas predefinidas para llamadas específicas realizadas durante una prueba. Los stubs se utilizan para simular el comportamiento de una dependencia real y controlar las respuestas que debe proporcionar en un escenario particular de prueba.



Tipos de Testing Double

Mock:

Es un objeto que tiene expectativas predefinidas sobre las interacciones que recibirá durante una prueba. Los mocks se utilizan para verificar que la unidad de código bajo prueba se comunica adecuadamente con sus dependencias, y se verifica que las interacciones esperadas realmente ocurran durante la prueba.

› JUnit y Mockito

¿QUÉ ES MOCKITO?

Mockito y los dobles

Mockito es una librería Java que permite simular el comportamiento de una clase de forma dinámica. De esta forma nos aislamos de las dependencias con otras clases y sólo testeamos la funcionalidad concreta que queremos.

¿QUÉ ES MOCKITO?

a simulación del comportamiento de una clase se hace mediante los “dobles” que pueden ser de distintos tipos:

- **Dummy**: Son objetos que se utilizan para realizar llamadas a otros métodos, pero no se usan.
- **Stub**: es como un dummy ya que sus métodos no hacen nada, pero devuelven cierto valor que necesitamos para ejecutar nuestro test con respecto a ciertas condiciones.
- **Spy**: Es un objeto real que permite verificar el uso que se hace del propio objeto, por ejemplo el número de veces que se ejecuta un método o los argumentos que se le pasan.
- **Mock**: Es un stub en el que sus métodos sí implementan un comportamiento, pues esperan recibir unos valores y en función de ellos devuelve una respuesta.
- **Fake**: Son objetos que tienen una implementación que funciona pero que no son apropiados para usar en producción (por ejemplo, una implementación de HttpSession).

PASOS PREVIOS EJEMPLO MOCKITO

Si nuestra aplicación Android usa Coroutines y LiveData, entonces necesitamos tener en cuenta la forma en que el proceso de test unitarios es llevado a cabo. Para mostrar su implementación en un proyecto “real”, continuaremos usando la app NASA Rover Photos, el proyecto en el cual hemos estado trabajando durante los artículos previos:

1. Paso a paso para usar Bottom Navigation junto al Navigation Component de Jetpack usando múltiples grafos de navegación
2. Implementando una app Android usando Jetpack, MVVM + UI State (... y otras cosillas interesantes)
3. Tests de UI en Android con Espresso e Idling Resources

https://github.com/cdmunoz/NasaRoverPhotos

AÑADIENDO DEPENDENCIAS

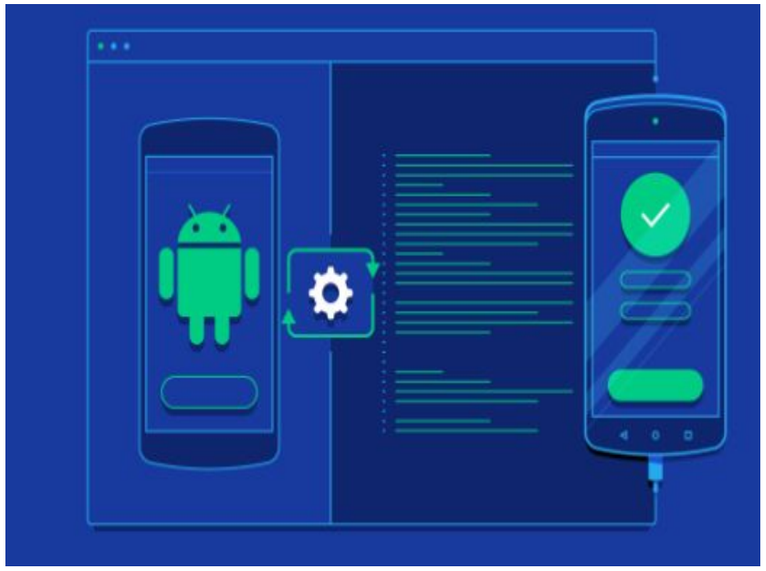
Adicionando Las Dependencias Requeridas
El primer paso es el de configurar las dependencias. Así que agregamos las de **Mockito** (para mockear objetos), las de **Arch Core** (para trabajar con LiveData) y las de **Coroutines Test** (para usar los Dispatchers adecuadamente):

```
def mockitoCoreVersion = "3.3.3"
testImplementation
"org.mockito:mockito-core:$mockitoCoreVersion
"
def archCoreTest = "2.1.0"
testImplementation
"androidx.arch.core:core-testing:$archCoreTest"
def coroutinesTest = "1.3.4"
testImplementation
"org.jetbrains.kotlinx:kotlinx-coroutines-test:$cor
outinesTest"
def mockWebserverVersion = "4.7.2"
testImplementation
"com.squareup.okhttp3:mockwebserver:$mock
WebserverVersion"
```

CREANDO UN TEST

Creando Una Test Coroutine Rule

Esta es una custom rule que nos permite setear y resetear el main dispatcher cuando ejecutamos un unit test que usa coroutinas. También podemos definir en ella un método que retorne un **runBlockingTest**, que permite tener el alcance del test ejecutándose en el hilo principal. Como este es un desarrollo experimental dentro del Api de coroutinas, el test debe ser marcado como tal (@ExperimentalCoroutinesApi). Recordemos que esta regla la creamos también para poder usarla fácilmente en diferentes tests en diferentes archivos.



› Creando Tests

CREANDO UN TEST

```
@ExperimentalCoroutinesApi
class TestCoroutineRule : TestRule {

    private val testCoroutineDispatcher = TestCoroutineDispatcher()
    private val testCoroutineScope = TestCoroutineScope(testCoroutineDispatcher)

    override fun apply(base: Statement?, description: Description?) = object : Statement() {
        @Throws(Throwable::class)
        override fun evaluate() {
            Dispatchers.setMain(testCoroutineDispatcher)
            base?.evaluate()
            Dispatchers.resetMain()
            testCoroutineScope.cleanupTestCoroutines()
        }
    }

    fun runBlockingTest(block: suspend TestCoroutineScope.() -> Unit) =
        testCoroutineScope.runBlockingTest { block() }

}
```

CREANDO EL TEST

Creando El Test
Cuando testeamos los ViewModels, como podemos ver, usamos la *InstantTaskExecutorRule* para indicar que estamos trabajando con LiveData y así forzamos ejecutarlo en el mismo hilo del test. **También usaremos la custom rule que acabamos de crear, TestCoroutineRule**. El resto es solo crear los mock requeridos (la clase repository y el observer para este caso en particular) y las lógicas de verificación. Cabe resaltar que cuando usamos coroutinas el test debe ser usado dentro del bloque ***runBlockingTest*** y su respectivo alcance (scope).

CREANDO EL TEST

```
1  @ExperimentalCoroutinesApi
2  @RunWith(MockitoJUnitRunner::class)
3  class PhotosViewModelTest {
4
5      @get:Rule
6      val testInstantTaskExecutorRule: TestRule = InstantTaskExecutorRule()
7      @get:Rule
8      val testCoroutineRule = TestCoroutineRule()
9      private lateinit var viewModel: PhotosViewModel
10     @Mock
11     private lateinit var photosRepository: PhotosRepository
12     @Mock
13     private lateinit var photosResponseObserver: Observer<Result<ArrayList<Photo>>>
14
15     @Before
16     fun setUp() {
17         viewModel = PhotosViewModel(photosRepository)
18     }
19
20     @Test
21     fun `when fetching results ok then return a list successfully`() {
22         val emptyList = arrayListOf<Photo>()
23         testCoroutineRule.runBlockingTest {
24             viewModel.getPhotos().observeForever(photosResponseObserver)
25             whenever(photosRepository.getPhotosFromApi(anyInt(), anyInt())).thenAnswer {
26                 Result.Success(emptyList)
27             }
28             viewModel.loadData()
29             assertNotNull(viewModel.getPhotos().value)
30             assertEquals(Result.Success(emptyList), viewModel.getPhotos().value)
31         }
32     }
33
34     @Test
35     fun `when calling for results then return loading`() {
36         testCoroutineRule.runBlockingTest {
37             viewModel.getPhotos().observeForever(photosResponseObserver)
38             viewModel.loadData()
39             verify(photosResponseObserver).onChanged(Result.InProgress)
40         }
41     }
42
43     @Test
44     fun `when fetching results fails then return an error`() {
45         val exception = mock(HttpException::class.java)
46         testCoroutineRule.runBlockingTest {
47             viewModel.getPhotos().observeForever(photosResponseObserver)
48             whenever(photosRepository.getPhotosFromApi(anyInt(), anyInt())).thenAnswer {
49                 Result.Error(exception)
50             }
51             viewModel.loadData()
52             assertNotNull(viewModel.getPhotos().value)
53             assertEquals(Result.Error(exception), viewModel.getPhotos().value)
54         }
55     }
56
57     @After
58     fun tearDown() {
59         viewModel.getPhotos().removeObserver(photosResponseObserver)
60     }
61 }
```

alkemy

TESTEANDO EL REPOSITORIO

Testeando El Repositorio
A much@s devs les gusta agregar unit test a sus clases Repository, aunque a much@s otr@s no, bajo la idea que dada la naturaleza de la clase repository al ser una clase container. De todas formas, podemos igual asegurarnos que estamos obteniendo los resultados esperados usando la librería MockWebServer de Square. Esta puede sernos de gran ayuda, pues permite mockear un API response usando un archivo JSON (con la estructura de la respuesta) que podemos crear localmente dentro de nuestro proyecto.

TESTEANDO EL REPOSITORIO

```
abstract class MockWebServerBaseTest {

    private lateinit var mockServer: MockWebServer

    @Before
    open fun setUp() {
        this.configureMockServer()
    }

    @After
    open fun tearDown() {
        this.stopMockServer()
    }

    abstract fun isMockServerEnabled(): Boolean

    open fun configureMockServer() {
        if (isMockServerEnabled()) {
            mockServer = MockWebServer()
            mockServer.start()
        }
    }

    open fun stopMockServer() {
        if (isMockServerEnabled()) {
            mockServer.shutdown()
        }
    }

    open fun mockHttpResponse(fileName: String, responseCode: Int) =
        mockServer.enqueue(MockResponse().setResponseCode(responseCode).setBody(getJson(fileName

    open fun mockHttpResponse(responseCode: Int) =
        mockServer.enqueue(MockResponse().setResponseCode(responseCode))

    private fun getJson(path: String): String {
        val uri = this.javaClass.classLoader!!.getResource(path)
        val file = File(uri.path)
        return String(file.readBytes())
    }

    fun provideTestApiService(): ApiService {
        return Retrofit.Builder().baseUrl(mockServer.url("/")).addConverterFactory(
            GsonConverterFactory.create())
            .client(OkHttpClient.Builder().build()).build().create(ApiService::class.java)
    }
```

TESTEANDO EL REPOSITORIO

Así, nuestro unit test para la repositorio sería algo como esto:

```
@RunWith(JUnit4::class)
class PhotosRepositoryTest : MockWebServerBaseTest() {

    @get:Rule
    val testInstantTaskExecutorRule: TestRule = InstantTaskExecutorRule()

    private lateinit var photosRepository: PhotosRepository
    private lateinit var apiService: ApiService
    private val sol = anyInt()
    private val page = anyInt()

    override fun isMockServerEnabled() = true

    @Before
    fun start() {
        apiService = provideTestApiService()
        photosRepository = PhotosRepository(apiService)
    }

    @Test
    fun `given response ok when fetching results then return a list with elements`() {
        runBlocking {
            mockHttpResponse("json/photos_response_one_item.json", HttpURLConnection.HTTP_OK)
            val apiResponse = photosRepository.getPhotosFromApi(sol, page)

            assertNotNull(apiResponse)
            assertEquals(apiResponse.extractData?.size, 1)
        }
    }

    @Test
    fun `given response ok when fetching empty results then return an empty list`() {
        runBlocking {
            mockHttpResponse("json/photos_response_empty_list.json", HttpURLConnection.HTTP_OK)
            val apiResponse = photosRepository.getPhotosFromApi(sol, page)

            assertNotNull(apiResponse)
            assertEquals(apiResponse.extractData?.size, 0)
        }
    }

    @Test
    fun `given response failure when fetching results then return exception`() {
        runBlocking {
            mockHttpResponse(PhotosRepository.GENERAL_ERROR_CODE)
            val apiResponse = photosRepository.getPhotosFromApi(sol, page)

            assertNotNull(apiResponse)
            val expectedValue = Result.Error(Exception(SOMETHING_WRONG))
            assertEquals(expectedValue.exception.message, (apiResponse as Result.Error).exceptic
        }
    }
```

› Anotaciones Junit

TESTEANDO EL REPOSITORIO

Lo más importante aquí es el uso del bloque nativo de coroutinas **runBlocking** en lugar de **runBlockingTest**. Y por qué no funciona con *runBlockingTest*? Aún no ha sido explicado completamente, pero en el thread de este issue están las posibles soluciones y sus explicaciones mientras el fix se implementa. El resto es puro *mockwebserver + mockito + asserts*.

Junit

JUnit es un framework Java para implementar test en Java. Se basa en anotaciones:

- **@Test**: indica que el método que la contiene es un test: expected y timeout.
- **@Before**: ejecuta el método que la contiene justo antes de cada test.
- **@After**: ejecuta el método que la contiene justo después de cada test.
- **@BeforeClass**: ejecuta el método (estático) que la contiene justo antes del **primer test**.
- **@AfterClass**: ejecuta el método (estático) que la contiene justo después del **último test**.
- **@Ignore**: evita la ejecución del tests. No es muy recomendable su uso porque puede ocultar test fallidos. Si dudamos si el test debe estar o no, quizás borrarlo es la mejor de las decisiones.

JUnit

¿Qué es?:

JUnit es un popular framework de pruebas unitarias para Java. Es una biblioteca de código abierto que proporciona clases y anotaciones que facilitan la creación y ejecución de pruebas unitarias de manera efectiva y organizada.
JUnit permite a los desarrolladores escribir pruebas automatizadas para verificar el correcto funcionamiento de unidades de código individuales, como métodos o clases, de forma independiente y repetitiva. Estas pruebas unitarias se enfocan en probar pequeñas partes aisladas del código, lo que facilita la detección temprana de errores y contribuye a un desarrollo más confiable y robusto.

Notaciones JUnit

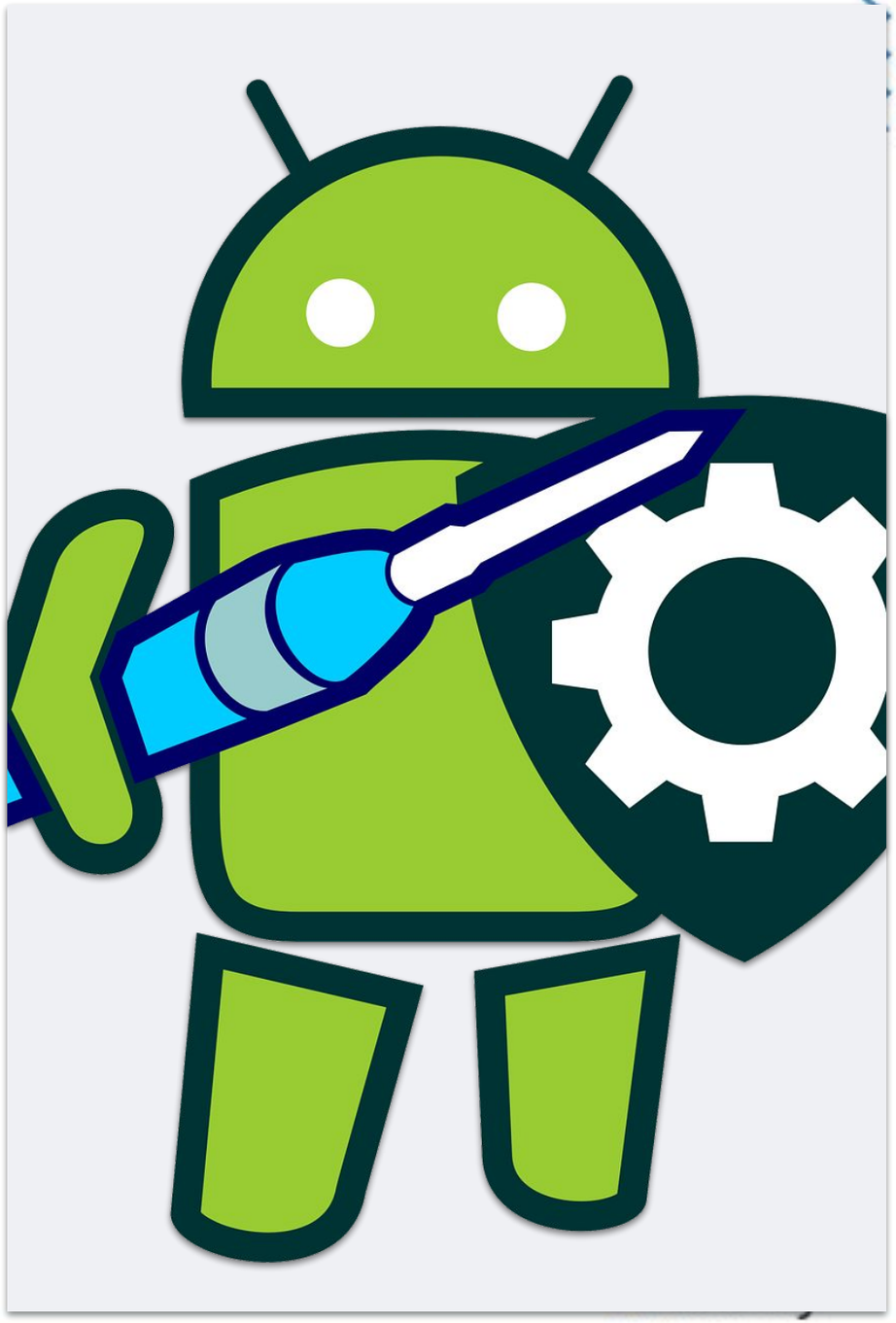
@Test:

Esta anotación se utiliza para marcar un método como una prueba unitaria. JUnit ejecutará todos los métodos anotados con @Test como pruebas individuales.

Notaciones JUnit

@Before:

Esta anotación se utiliza para marcar un método que se ejecutará antes de cada prueba. Se utiliza para configurar el entorno necesario para las pruebas, como inicializar objetos o recursos.



Notaciones JUnit

@After:

Esta anotación se utiliza para marcar un método que se ejecutará después de cada prueba. Se utiliza para liberar recursos o limpiar el entorno después de la prueba.

LIVE CODING

Ejemplo en vivo

Analizando Pruebas

Para esta Actividad , analizaremos un caso previo ya creado. EL cual tiene sus clases testing ya Incorporados. La idea es Sacar cosas en limpio, como por ej : Implementación, creación de clases , analizar código, y ver el comportamiento de nuestra clases juntas con nuestra aplicación.

Código Base :**https://github.com/Mariocanedo/CASOFUTURO**

Tiempo: 1 hr.

alkemy

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ *Testing y control de calidad*
- ✓ *Tipos de Testing*
- ✓ *Configuración de Testing*
- ✓ *Tests Unitarios*
- ✓ *Test Double*
- ✓ *Usos de JUnit*

< ¡Muchas gracias! >

alkemy